UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

Facultad de Ingeniería de Producción y Servicios

Escuela Profesional de Ingeniería de Sistemas

Sistemas Operativos

Semestre VI – Periodo 2026-B

PRÁCTICA DOMICILIARIA – SEMANA 7

Servicio concurrente de reservas

Sincronización, exclusión mutua, espera con timeout y cancelación

INFORME TÉCNICO

Servicio concurrente sobre M recursos con ocho o más workers, invariantes, prueba de carrera y medición reproducible

AUTOR

Gustavo Alonso Carrillo Villalta

PERIODO ACADÉMICO

2026-B

DOCENTE

Norman Patrick Harvey Arce

Arequipa – Perú

2026

1. Resumen

Se presenta un servicio concurrente de reservas implementado en Python 3. Cada recurso admite como máximo una reserva activa. El diseño permite consultar disponibilidad, reservar, cancelar, esperar con timeout y cerrar el servicio sin dejar workers bloqueados. Un mutex protege el estado y una variable de condición permite suspender a los solicitantes mientras el recurso está ocupado. El informe incluye una versión defectuosa cuyo intercalado se fuerza mediante una barrera, pruebas automatizadas y un generador de métricas. Los tiempos concretos dependen del equipo y se obtienen ejecutando los scripts incluidos; no se presuponen resultados medidos.

2. Problema y objetivos

El servicio administra M recursos discretos. Múltiples threads pueden solicitar el mismo recurso, pero una reserva solo se concede si este se encuentra libre. Las operaciones sobre el estado compartido deben ser indivisibles desde el punto de vista de los demás workers. La implementación se ajusta a la distinción de la semana 7 entre exclusión mutua, progreso y espera acotada.

2.1. Requisitos funcionales

- Consultar los recursos disponibles.
- Reservar un recurso libre sin duplicar la asignación.
- Cancelar una reserva existente y notificar a los que esperan.
- Esperar hasta que un recurso se libere, se alcance el timeout o se cierre el servicio.
- Cerrar el servicio y despertar a todos los esperadores.
- Registrar operaciones y duración de esperas.

3. Modelo de estado e invariantes

Sea $R = \{0, \ldots, M-1\}$ el conjunto de recursos y B el mapa de reservas activas. Cada entrada relaciona un recurso con un identificador de reserva único y un cliente.

- I_1: $0 \leq |B| \leq M$.
- I_2: cada recurso aparece como máximo una vez en B.
- I_3: los identificadores de reserva activos son únicos.
- I_4: $disponibles = M - |B|$.
- I_5: después de cerrar, ningún nuevo intento de reserva obtiene éxito.

La transición crítica es comprobar disponibilidad y, dentro de la misma sección protegida, insertar la reserva y asignar su identificador. Cancelar elimina la reserva bajo el mismo lock y luego notifica a los esperadores.

4. Diseño de sincronización

Primitiva	Justificación
`threading.Lock`	Protege el mapa, el contador de identificadores, el cierre y las métricas; evita que dos workers aprueben simultáneamente una misma reserva.
`threading.Condition`	Asocia una condición al mismo lock. `wait(timeout)` libera el lock mientras espera y lo vuelve a adquirir antes de continuar.
`notify_all()`	Se usa tras cancelación o cierre. Cada worker vuelve a comprobar el predicado dentro de un bucle, porque un despertar no garantiza que el recurso siga libre.

No se usa un semáforo como sustituto del estado por recurso: el predicado verdadero es si el recurso está o no en el mapa de reservas. El timeout limita la duración de la espera de un solicitante, pero no sustituye la sincronización.

5. Implementación y ejecución

El archivo `reservas.py` contiene el servicio, la demostración defectuosa y la carga de ocho workers. El método `esperar_y_reservar` calcula una fecha límite monotónica y recalcula el tiempo restante después de cada despertar. Las entradas inválidas generan `ValueError`; las solicitudes que no pueden reservarse devuelven `None`.

```
python3 --version
python3 -m unittest -v
python3 reservas.py
```

5.1. Terminación limpia

El método `cerrar()` cambia el estado bajo el lock y llama a `notify_all()`. Los esperadores detectan el cierre y retornan. En el programa de pruebas, todos los threads se unen mediante `join`; por tanto, el proceso principal no abandona workers activos.

6. Carrera reproducible en la versión defectuosa

La clase `ServicioDefectuoso` separa la lectura de disponibilidad de la escritura y no protege la transición. Una barrera de dos participantes hace que ambos threads lean

`ocupado=False` antes de que cualquiera escriba. Ambos devuelven éxito y el contador de reservas llega a dos aunque solo existe un recurso.

```
Carrera defectuosa (reservas aceptadas): 2
```

Este resultado es intencional y determinista para la demostración. En el servicio corregido, la comprobación y la escritura están dentro de la misma sección crítica, por lo que como máximo uno de los ocho workers puede obtener la reserva del único recurso.

7. Plan de pruebas

ID	Escenario	Resultado esperado
T1	Ocho workers intentan reservar un recurso a la vez	Exactamente una reserva exitosa.
T2	Cancelar una reserva válida	El recurso vuelve a estar disponible.
T3	Espera con timeout mientras el recurso sigue ocupado	Retorna `None` al vencer el plazo y registra espera.
T4	Un worker espera y otro cancela	El esperador despierta y puede reservar.
T5	Cierre con un worker bloqueado en espera	El worker despierta y termina.
T6	Recurso fuera de rango o timeout negativo	Error controlado y estado sin cambios.
T7	Carrera defectuosa con barrera	Dos éxitos para un recurso; demuestra el fallo.

Las pruebas están implementadas en `test_reservas.py`. Se ejecutan con `python3 -m unittest -v`. El test de carrera espera exactamente dos reservas defectuosas; los demás verifican las propiedades del servicio corregido.

8. Métricas de espera y rendimiento

El método `demo_carga(8,100)` intenta 800 operaciones con ocho workers y cuatro recursos. Informa duración total, operaciones intentadas por segundo, número de operaciones por tipo, cantidad de muestras de espera y esperas media y máxima. Estas son métricas de una carga sintética, no una predicción del rendimiento de producción.

```
python3 reservas.py
```

Para comparar ejecuciones, registrar la versión de Python, sistema operativo, número de workers, número de recursos y repetir el comando varias veces. Reportar mediana y rango; no comparar cifras obtenidas con cargas distintas. El rendimiento depende de la contención, la planificación de threads y el hardware.

9. Complejidad y limitaciones

La consulta de disponibilidad cuesta $O(M)$; reservar un recurso mediante el mapa tiene costo promedio $O(1)$; cancelar cuesta $O(M)$ en esta implementación porque busca el identificador recorriendo las reservas. La memoria ocupada es $O(M)$. `notify_all()` puede despertar a más workers de los necesarios; para cargas mayores podría usarse una condición por recurso. La solución no persiste reservas ante una caída ni autentica clientes; es un prototipo docente de sincronización.

10. Conclusiones

1. La decisión de disponibilidad y la creación de la reserva deben pertenecer a una única sección crítica.
2. El mutex conserva los invariantes; la condición permite esperar sin consumir CPU mediante espera activa.
3. Los timeouts deben usar una fecha límite y un bucle que vuelva a comprobar el predicado.
4. El cierre necesita notificar a todos los esperadores y comprobar que los workers finalicen.
5. La carrera forzada evidencia por qué una comprobación aislada no impide reservas duplicadas.

11. Referencias

1. A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ: Wiley, 2018.
2. Python Software Foundation, “threading — Thread-based parallelism”. Disponible en: https://docs.python.org/3/library/threading.html.
3. Python Software Foundation, “unittest — Unit testing framework”. Disponible en: https://docs.python.org/3/library/unittest.html.

A. Código fuente completo

A.1. Implementación: `reservas.py`

```python
import threading
import time
from collections import Counter
from concurrent.futures import ThreadPoolExecutor

class ServicioReservas:
    """Servicio concurrente: cada recurso admite como maximo una reserva."""
    def __init__(self, cantidad):
        if not isinstance(cantidad, int) or cantidad <= 0:
            raise ValueError("cantidad debe ser un entero positivo")
        self.cantidad = cantidad
        self.reservas = {}                # recurso -> identificador de reserva
        self._lock = threading.Lock()
        self._cond = threading.Condition(self._lock)
        self._cerrado = False
        self._siguiente_id = 1
        self._esperas = []
        self._operaciones = Counter()

    def _validar_recurso(self, recurso):
        if not isinstance(recurso, int) or not 0 <= recurso < self.cantidad:
            raise ValueError(f"recurso fuera de rango: {recurso}")

    def consultar(self):
        with self._lock:
            self._operaciones["consulta"] += 1
            return [i for i in range(self.cantidad) if i not in self.reservas]

    def reservar(self, recurso, cliente):
        self._validar_recurso(recurso)
        with self._cond:
            if self._cerrado:
                return None
            if recurso in self.reservas:
                self._operaciones["rechazo_ocupado"] += 1
                return None
            reserva_id = self._siguiente_id
            self._siguiente_id += 1
            self.reservas[recurso] = (reserva_id, cliente)
            self._operaciones["reserva"] += 1
            self._comprobar_invariantes()
            return reserva_id

    def esperar_y_reservar(self, recurso, cliente, timeout):
        self._validar_recurso(recurso)
        if timeout is not None and timeout < 0:
            raise ValueError("timeout no puede ser negativo")
        inicio = time.perf_counter()
        limite = None if timeout is None else inicio + timeout
        with self._cond:
            while recurso in self.reservas and not self._cerrado:
                restante = None if limite is None else limite - time.perf_counter()
                if restante is not None and restante <= 0:
                    self._esperas.append(time.perf_counter() - inicio)
                    self._operaciones["timeout"] += 1
                    return None
                self._cond.wait(restante)
            self._esperas.append(time.perf_counter() - inicio)
            if self._cerrado:
                self._operaciones["cancelada_cierre"] += 1
                return None
            reserva_id = self._siguiente_id
            self._siguiente_id += 1
            self.reservas[recurso] = (reserva_id, cliente)
            self._operaciones["reserva_tras_espera"] += 1
```

```
            self._comprobar_invariantes()
            return reserva_id

    def cancelar(self, reserva_id):
        with self._cond:
            for recurso, dato in list(self.reservas.items()):
                if dato[0] == reserva_id:
                    del self.reservas[recurso]
                    self._operaciones["cancelacion"] += 1
                    self._comprobar_invariantes()
                    self._cond.notify_all()
                    return True
            self._operaciones["cancelacion_no_encontrada"] += 1
            return False

    def cerrar(self):
        with self._cond:
            self._cerrado = True
            self._cond.notify_all()

    def estado(self):
        with self._lock:
            return {"capacidad": self.cantidad,
                    "reservados": len(self.reservas),
                    "disponibles": self.cantidad - len(self.reservas),
                    "reservas": dict(self.reservas),
                    "cerrado": self._cerrado}

    def metricas(self):
        with self._lock:
            esperas = sorted(self._esperas)
            n = len(esperas)
            return {"operaciones": dict(self._operaciones),
                    "muestras_espera": n,
                    "espera_media_s": sum(esperas)/n if n else 0.0,
                    "espera_max_s": max(esperas) if n else 0.0}

    def _comprobar_invariantes(self):
        assert 0 <= len(self.reservas) <= self.cantidad
        assert len({x[0] for x in self.reservas.values()}) == len(self.reservas)
        assert self.cantidad - len(self.reservas) >= 0


# Demostracion defectuosa: lectura y escritura separadas sin exclusion mutua.
class ServicioDefectuoso:
    def __init__(self):
        self.ocupado = False
        self.reservas = 0
    def reservar_con_barrera(self, barrera):
        disponible = not self.ocupado
        barrera.wait(timeout=2)
        if disponible:
            self.ocupado = True
            self.reservas += 1
            return True
        return False

def demo_carrera():
    barrera = threading.Barrier(2)
    servicio = ServicioDefectuoso()
    resultados = []
    def worker():
        resultados.append(servicio.reservar_con_barrera(barrera))
    ts = [threading.Thread(target=worker) for _ in range(2)]
    for t in ts: t.start()
    for t in ts: t.join()
    return servicio.reservas, resultados

def demo_carga(workers=8, operaciones_por_worker=100):
    servicio = ServicioReservas(4)
    inicio = time.perf_counter()
    def worker(w):
```

```
        for i in range(operaciones_por_worker):
            recurso = (w + i) % 4
            rid = servicio.reservar(recurso, f"W{w}")
            if rid is not None:
                servicio.cancelar(rid)
            else:
                servicio.consultar()
    with ThreadPoolExecutor(max_workers=workers) as pool:
        list(pool.map(worker, range(workers)))
    duracion = time.perf_counter() - inicio
    return {"workers": workers, "operaciones_intentadas": workers*operaciones_por_worker,
            "duracion_s": duracion,
            "ops_s": workers*operaciones_por_worker/duracion if duracion else 0,
            "estado_final": servicio.estado(), "metricas": servicio.metricas()}

if __name__ == "__main__":
    print("Carrera defectuosa (reservas aceptadas):", demo_carrera()[0])
    print("Carga concurrente:", demo_carga())
```

A.2. Pruebas automatizadas: `test_reservas.py`

```
import threading
import unittest
from reservas import ServicioReservas, demo_carrera

class TestReservas(unittest.TestCase):
    def test_no_doble_reserva(self):
        s = ServicioReservas(1)
        barrera = threading.Barrier(8)
        resultados = []
        def w(i):
            barrera.wait()
            resultados.append(s.reservar(0, f"W{i}"))
        ts = [threading.Thread(target=w, args=(i,)) for i in range(8)]
        for t in ts: t.start()
        for t in ts: t.join()
        self.assertEqual(sum(x is not None for x in resultados), 1)
        self.assertEqual(s.estado()["reservados"], 1)

    def test_cancelacion_libera_recurso(self):
        s = ServicioReservas(1)
        rid = s.reservar(0, "A")
        self.assertTrue(s.cancelar(rid))
        self.assertEqual(s.consultar(), [0])

    def test_timeout(self):
        s = ServicioReservas(1)
        s.reservar(0, "A")
        self.assertIsNone(s.esperar_y_reservar(0, "B", 0.02))
        self.assertGreaterEqual(s.metricas()["espera_max_s"], 0.01)

    def test_espera_se_despierta_tras_cancelar(self):
        s = ServicioReservas(1)
        rid = s.reservar(0, "A")
        resultado = []
        t = threading.Thread(target=lambda: resultado.append(s.esperar_y_reservar(0, "B", 1.0)))
        t.start()
        s.cancelar(rid)
        t.join(1.5)
        self.assertFalse(t.is_alive())
        self.assertIsNotNone(resultado[0])

    def test_cierre_despierta_esperadores(self):
        s = ServicioReservas(1)
        s.reservar(0, "A")
        resultado = []
        t = threading.Thread(target=lambda: resultado.append(s.esperar_y_reservar(0, "B", 5))
        )
        t.start()
```

```
        s.cerrar()
        t.join(1)
        self.assertFalse(t.is_alive())
        self.assertIsNone(resultado[0])

    def test_datos_invalidos(self):
        s = ServicioReservas(2)
        with self.assertRaises(ValueError): s.reservar(4, "A")
        with self.assertRaises(ValueError): s.esperar_y_reservar(0, "B", -1)
        self.assertEqual(s.estado()["reservados"], 0)

    def test_carrera_defectuosa_reproducible(self):
        reservas, _ = demo_carrera()
        self.assertEqual(reservas, 2)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```